

UCS503 – Software Engineering Project
Academic Year 2026–2027

Blynk: A Browser-Based Eye Strain and Posture Monitor

Privacy-First, On-Device Wellness Tracking

Submitted by:

1024030442 Hardik Chandna

B.E. Third Year, Computer Engineering

Submitted to:

Mr Jeelani Asif

**Department of Computer Science and
Engineering**

Mid-Semester Evaluation Report

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1. Introduction	2
1.1. Background and Context	2
1.1.1. Problem Statement	2
1.2. Project Objectives	2
2. Methodology and Design	4
2.1. System Architecture	4
2.1.1. High-Level Design	4
2.1.2. Technical Stack	4
2.2. Implementation Details	5
2.2.1. Module 1: Core Detection Pipeline	5
2.2.2. Module 2: Planned Feature Set	5
3. Results and Evaluation	6
3.1. Testing Methodology	6
3.2. Performance Metrics	6
3.3. Analysis	7
4. Conclusion	8
4.1. Summary of Work	8
4.2. Key Findings	8
4.3. Future Work and Recommendations	8
4.4. Final Remarks	9

Chapter 1

Introduction

1. Background and Context

Long screen sessions are now a routine part of both academic and professional life, and they carry a well-documented physical cost. When a person focuses on a screen, blink rate drops significantly below normal, which dries out and strains the eyes over time – a pattern commonly referred to as Computer Vision Syndrome, or digital eye strain. Prolonged, forward-leaning screen posture compounds this with chronic neck and shoulder strain.

The tools currently available to address this are largely passive. Break-reminder applications fire a fixed timer at regular intervals, but have no way of confirming whether the user actually rested their eyes or simply dismissed the notification and continued working. Tools that do attempt to measure something meaningful, such as fatigue or attention, typically require uploading a video feed to an external server for processing, which raises real privacy concerns for a camera pointed at a person’s face for hours at a time.

1.1. Problem Statement

There is no lightweight, private, real-time system that both measures eye strain and screen distance directly, and verifies that a recommended recovery action was genuinely completed, rather than simply reminding the user and trusting them to comply. Blynk addresses this gap through an entirely browser-based application: facial landmark detection runs on-device, blink rate and screen distance are calculated locally, and a break is only marked complete once the system has verified, through live eye tracking, that the user actually followed the prescribed recovery exercise.

2. Project Objectives

The core objectives of the Blynk project comprise:

- **On-device detection:** Track blink rate and estimated screen distance in real time from the webcam feed, with all processing happening on the user’s own device.
- **Live feedback:** Present this data through a readable dashboard, so the user has an ongoing sense of their eye strain without needing to interpret raw numbers.
- **Verified recovery:** Implement a break-enforcement feature that confirms, through live gaze tracking, that a recovery exercise was genuinely completed before allowing the user to continue
- **Local history:** Store daily and weekly usage trends entirely on the user’s device, with no external database involved.
- **Privacy by architecture:** Ensure no camera frame or video data is ever transmitted off the user’s device, verifiable directly through browser network inspection.

Chapter 2

Methodology and Design

1. System Architecture

1.1. High-Level Design

Blynk runs entirely client-side, with no backend server required for any core feature. The architecture is organized into three layers: an on-device sensing and inference layer, an analysis layer that turns raw landmark data into meaningful metrics, and a presentation layer that displays those metrics and drives the break-enforcement flow.

Presentation Layer React dashboard, Framer Motion animation, Three.js ambient visual, D3.js history charts
Analysis Layer Blink detection (eye aspect ratio), screen distance estimation, break verification logic
Sensing and Inference Layer Browser webcam (<code>getUserMedia</code>), MediaPipe Face Landmarker (WebAssembly, on-device)

The workflow is straightforward: the browser requests camera access, MediaPipe's pretrained Face Landmarker model processes each frame to produce 468 facial landmark points, the analysis layer derives blink rate and distance from those points, and the presentation layer renders the result and triggers the break-enforcement sequence when sustained strain is detected.

1.2. Technical Stack

- **Inference:** MediaPipe Face Landmarker, running in-browser via WebAssembly for near-instant, on-device processing.
- **Frontend framework:** React, bundled with Vite.
- **Camera access:** the browser-native `getUserMedia` API.
- **Planned additions:** Framer Motion (animation), Three.js via react-three-fiber (ambient visual), D3.js (history charts), IndexedDB (local storage).
- **Development environment:** VS Code, Node.js, npm.
- **Version control:** Git, with the project hosted on GitHub and documented via a GitHub Pages site.

2. Implementation Details

2.1. Module 1: Core Detection Pipeline

This module is the foundation the rest of the application depends on, and is the primary implementation work completed so far.

- **Camera capture:** live webcam access implemented via `getUserMedia`, feeding frames continuously into the detection pipeline.
- **Facial landmark detection:** MediaPipe's pretrained Face Landmarker model loaded and run in-browser, producing 468 landmark points per frame in real time, rendered as a live overlay for verification during development.
- **Blink detection:** implemented using the eye aspect ratio (EAR) technique – the ratio of vertical to horizontal distance between specific eyelid landmark points. A threshold-crossing state machine detects each blink event and maintains a running blink count.

2.2. Module 2: Planned Feature Set

The following features build on Module 1 and are the focus of the second half of the project timeline.

- **Screen distance calibration:** a one-time calibration step recording a baseline interpupillary distance, against which live distance is measured.
- **Live dashboard:** animated gauges reflecting blink rate and distance in real time.
- **Break enforcement:** a full-screen overlay presenting a moving target, with live gaze verification determining whether the screen unlocks.
- **Local history:** periodic metric snapshots stored in IndexedDB, surfaced through trend charts.

Chapter 3

Results and Evaluation

1. Testing Methodology

Since Blynk is a personal, single-user wellness tool rather than a system with a large user base, testing at this stage has focused on direct, repeated self-testing rather than formal load or accuracy benchmarking against a large dataset. The core detection pipeline was tested by running the live camera feed and landmark overlay across a range of conditions: normal indoor lighting, low light, with and without glasses, and at different times of day. Blink detection was validated by comparing the live blink counter against a manually counted reference while deliberately blinking at a controlled pace.

2. Performance Metrics

Formal performance evaluation – frame-rate benchmarking under sustained use, blink-detection accuracy against a larger, varied test set, and distance-calibration consistency across multiple users – has not yet been carried out, since these depend on features (distance calibration, break enforcement) that are still in progress. Table 3.1 summarizes the metrics planned for the next evaluation phase.

Evaluation Metric	Target Specification	Status
Blink detection accuracy vs. manual count	Within a small margin under normal lighting	Pending
Distance estimation consistency across repeated tests	Same warning at same physical distance	Pending
Real-time frame processing rate	Sustains browser responsiveness	Pending
Break verification accuracy	Correctly distinguishes genuine gaze-following	Pending
Confirmed absence of transmitted camera data	Zero outgoing requests carrying video	Confirmed

3. Analysis

At the current stage, the core sensing pipeline is functional and reliable in normal conditions: the webcam feed is captured correctly, facial landmarks track accurately across the visible frame, and blink detection using the eye aspect ratio technique produces a consistent, incrementing count during manual testing. This confirms that the underlying detection approach, built on a pretrained model rather than a custom-trained one, is sound for this application. The primary remaining engineering work is building the distance calibration step, the live dashboard, and the break-enforcement feature on top of this already-working foundation, followed by the accuracy and performance evaluation described above.

Chapter 4

Conclusion

1. Summary of Work

During the first half of the project timeline, the complete system modelling deliverables were produced: the project proposal, a UML use case diagram, data flow diagrams through Level 2, an entity-relationship diagram, an activity/swimlane diagram, and a Gantt chart, all published through the project's GitHub Pages site. On the implementation side, the core detection pipeline was built and validated: live webcam capture, real-time facial landmark detection using MediaPipe's pretrained model, and blink detection using the eye aspect ratio technique.

2. Key Findings

- A pretrained facial landmark model is sufficient for reliable, real-time blink detection without requiring any custom model training, confirming the architectural decision made at the proposal stage.
- Running inference entirely in-browser via WebAssembly keeps the application genuinely private by construction, not by policy, since no server exists in the pipeline to receive camera data.
- Direct, repeated self-testing across varied lighting conditions was an effective way to validate detection reliability at this stage, given the single-user nature of the tool.

3. Future Work and Recommendations

For the remainder of the project, the following work is planned:

- Implement screen-distance calibration and integrate it with the live dashboard.
- Build the break-enforcement feature, including the moving-target exercise and live gaze verification.
- Add local history storage via IndexedDB and trend visualization with D3.js.
- Carry out the performance and accuracy evaluation outlined in Section 3.2 once the remaining features are complete.
- Explore stretch goals, including posture detection using a second pretrained landmark model, and a native desktop widget version.

4. Final Remarks

Blynk addresses a real and common problem – the quiet, cumulative damage long screen sessions do to eyes and posture – with an approach that is both technically sound and genuinely private by design. The work completed so far demonstrates that the core detection pipeline is reliable and ready to support the remaining features. The rest of the project focuses on building the user-facing recovery mechanism and validating the complete system against the metrics defined above.